

Intermediate NumPy & Pandas Coding Questionnaire

Data Manipulation and Analytics

NumPy Section

Question 1: Matrix Slicing and Submatrix Extraction

Difficulty: Intermediate

Create a 6×6 matrix with values from 0 to 35. Extract the central 4×4 submatrix.

Solution:

```
import numpy as np

mat = np.arange(36).reshape(6, 6)
center = mat[1:5, 1:5]
print(center)
```

Expected Output:

```
[[ 7  8  9 10]
 [13 14 15 16]
 [19 20 21 22]
 [25 26 27 28]]
```

Question 2: Percentile-Based Filtering

Difficulty: Intermediate

Generate a random 1D array of 20 elements. Replace all values below the 25th percentile with zero.

Solution:

```
import numpy as np

arr = np.random.randn(20)
q25 = np.percentile(arr, 25)
arr[arr < q25] = 0
print(arr)
```

Expected Output: (varies due to randomness)

```
[-0.    0.    0.25  0.    0.15  0.42  0.    0.33  0.19  0.  
 0.48  0.    0.12  0.39  0.    0.26  0.41  0.    0.18  0.35]
```

Question 3: Min-Max Normalization

Difficulty: Intermediate

Given an array of integers, normalize all values between 0 and 1 using min-max normalization.

Solution:

```
import numpy as np  
  
data = np.random.randint(10, 100, size=12)  
norm = (data - np.min(data)) / (np.max(data) - np.min(data))  
print("Original:", data)  
print("Normalized:", norm)
```

Expected Output: (varies due to randomness)

```
Original: [45 82 23 67 91 34 78 56 89 42 71 35]  
Normalized: [0.29 0.84 0.    0.6  1.    0.13 0.75 0.43 0.97 0.26 0.68 0.14]
```

Question 4: 3D Array Stacking

Difficulty: Intermediate

Stack two random matrices of shape (4,3) and (4,3) along a new third dimension.

Solution:

```
import numpy as np  
  
a = np.random.rand(4, 3)  
b = np.random.rand(4, 3)  
stacked = np.stack([a, b], axis=2)  
print("Shape of stacked array:", stacked.shape)  
print("First element:\n", stacked[:, :, 0])
```

Expected Output:

```
Shape of stacked array: (4, 3, 2)  
First element:  
[[0.23 0.45 0.67]  
 [0.12 0.89 0.34]]
```

```
[0.56 0.78 0.42]
[0.91 0.11 0.64]]
```

Question 5: Unique Value Frequency Count

Difficulty: Intermediate

For a random array of integers, count unique values and their frequency efficiently.

Solution:

```
import numpy as np

arr = np.random.randint(1, 10, size=50)
values, counts = np.unique(arr, return_counts=True)
frequency_dict = dict(zip(values, counts))
print("Unique values and frequencies:")
print(frequency_dict)
```

Expected Output: (varies due to randomness)

```
Unique values and frequencies:
{1: 8, 2: 5, 3: 7, 4: 6, 5: 8, 6: 4, 7: 6, 8: 3, 9: 4}
```

Pandas Section

Question 6: Monthly Aggregation from Weekly Data

Difficulty: Intermediate

Given a sales DataFrame with 'Date', 'Product', 'Revenue', calculate monthly total revenue.

Solution:

```
import pandas as pd

df = pd.DataFrame({
    'Date': pd.date_range(start='2025-01-01', periods=12, freq='W'),
    'Product': ['A', 'B', 'C'] * 4,
    'Revenue': [100, 200, 150, 120, 220, 130, 110, 180, 170, 115, 235, 175]
})

df['Month'] = df['Date'].dt.to_period('M')
monthly_revenue = df.groupby('Month')['Revenue'].sum()
print(monthly_revenue)
```

Expected Output:

```
Month
2025-01    450
2025-02    350
2025-03    405
2025-04    350
Freq: M, Name: Revenue, dtype: int64
```

Question 7: Outlier Replacement with Mean

Difficulty: Intermediate

Replace outlier values in the 'Revenue' column (>95th percentile) with the mean revenue.

Solution:

```
import pandas as pd

df = pd.DataFrame({
    'Revenue': [100, 200, 150, 120, 220, 130, 110, 1000, 180, 170]
})

mean_rev = df['Revenue'].mean()
q95 = df['Revenue'].quantile(0.95)
print(f"95th Percentile: {q95}")
print(f"Mean: {mean_rev}")

df.loc[df['Revenue'] > q95, 'Revenue'] = mean_rev
print("\nDataFrame after outlier replacement:")
print(df)
```

Expected Output:

```
95th Percentile: 215.0
Mean: 237.0
```

DataFrame after outlier replacement:

	Revenue
0	100
1	200
2	150
3	120
4	220
5	130
6	110
7	237
8	180
9	170

Question 8: Pivot Table with Multiple Aggregations

Difficulty: Intermediate

Pivot the sales DataFrame so that months are rows and products are columns, showing total revenue for each cell.

Solution:

```
import pandas as pd

df = pd.DataFrame({
    'Date': pd.date_range(start='2025-01-01', periods=12, freq='W'),
    'Product': ['A', 'B', 'C'] * 4,
    'Revenue': [100, 200, 150, 120, 220, 130, 110, 180, 170, 115, 235, 175]
})

df['Month'] = df['Date'].dt.to_period('M')
pivot = pd.pivot_table(df, index='Month', columns='Product', values='Revenue', aggfunc='sum')
print(pivot)
```

Expected Output:

Product	A	B	C
Month			
2025-01	100	200.0	150.0
2025-02	120	220.0	130.0
2025-03	110	180.0	170.0
2025-04	115	235.0	175.0

Question 9: Fill Missing Values with Median

Difficulty: Intermediate

Given a DataFrame with missing values, fill them with the median of their respective columns.

Solution:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'A': [10, 20, 30, 40, 50],
    'B': [100, 200, 300, 400, 500],
    'C': [1.5, 2.5, 3.5, 4.5, 5.5]
})

# Introduce missing values
df.loc[2:2, 'A'] = np.nan
df.loc[1:2, 'B'] = np.nan
```

```
print("Before filling:")
print(df)

df_filled = df.fillna(df.median(numeric_only=True))

print("\nAfter filling with median:")
print(df_filled)
```

Expected Output:

```
Before filling:
   A      B      C
0 NaN  100.0  1.5
1 20.0   NaN  2.5
2 NaN  300.0  3.5
3 40.0   NaN  4.5
4 NaN  500.0  5.5

After filling with median:
   A      B      C
0 20.0  100.0  1.5
1 20.0  300.0  2.5
2 20.0  300.0  3.5
3 40.0  300.0  4.5
4 20.0  500.0  5.5
```

Question 10: Top N Values by Group

Difficulty: Intermediate

For a large DataFrame with repetitive 'Product' values, find the top 2 most profitable products by total profit.

Solution:

```
import pandas as pd

profit_df = pd.DataFrame({
    'Product': ['A', 'B', 'C', 'A', 'B', 'C', 'A', 'B'],
    'Profit': [40, 70, 60, 55, 85, 75, 65, 60]
})

top2 = profit_df.groupby('Product')['Profit'].sum().nlargest(2)
print("Top 2 most profitable products:")
print(top2)
```

Expected Output:

```
Top 2 most profitable products:
Product
B      215
```

```
A    160
Name: Profit, dtype: int64
```

Question 11: Inner Join on Key Column

Difficulty: Intermediate

Merge two DataFrames on 'Product', keeping only rows present in both (inner join).

Solution:

```
import pandas as pd

df1 = pd.DataFrame({
    'Product': ['A', 'B', 'C'],
    'Stock': [100, 150, 120]
})

df2 = pd.DataFrame({
    'Product': ['B', 'C', 'D'],
    'Sales': [95, 80, 125]
})

merged = pd.merge(df1, df2, on='Product', how='inner')
print("Inner join result:")
print(merged)
```

Expected Output:

```
Inner join result:
  Product  Stock  Sales
0       B    150     95
1       C    120     80
```

Question 12: Rolling Window Calculations

Difficulty: Intermediate

Calculate the rolling mean of 'Revenue' over each consecutive 3 rows.

Solution:

```
import pandas as pd

df = pd.DataFrame({
    'Date': pd.date_range(start='2025-01-01', periods=10, freq='D'),
    'Revenue': [100, 120, 115, 130, 140, 135, 150, 145, 160, 155]
})
```

```
df['RollingMean'] = df['Revenue'].rolling(window=3).mean()
print(df[['Date', 'Revenue', 'RollingMean']])
```

Expected Output:

	Date	Revenue	RollingMean
0	2025-01-01	100	NaN
1	2025-01-02	120	NaN
2	2025-01-03	115	111.67
3	2025-01-04	130	121.67
4	2025-01-05	140	128.33
5	2025-01-06	135	135.00
6	2025-01-07	150	141.67
7	2025-01-08	145	143.33
8	2025-01-09	160	151.67
9	2025-01-10	155	153.33

Question 13: Conditional Aggregation by Group

Difficulty: Intermediate

For each 'Product', compute the min and max 'Revenue' and add as new columns to the original DataFrame.

Solution:

```
import pandas as pd

df = pd.DataFrame({
    'Product': ['A', 'B', 'A', 'C', 'B', 'A'],
    'Revenue': [100, 200, 150, 120, 220, 130]
})

print("Original DataFrame:")
print(df)

min_max = df.groupby('Product')['Revenue'].agg(['min', 'max'])
df = df.merge(min_max, left_on='Product', right_index=True)

print("\nDataFrame with min/max columns:")
print(df)
```

Expected Output:

	Product	Revenue
0	A	100
1	B	200
2	A	150
3	C	120
4	B	220

5	A	130
---	---	-----

DataFrame with min/max columns:

	Product	Revenue	min	max
0	A	100	100	150
1	B	200	200	220
2	A	150	100	150
3	C	120	120	120
4	B	220	200	220
5	A	130	100	150

Question 14: String Filtering with Multiple Conditions

Difficulty: Intermediate

Extract all rows where 'Product' starts with 'A' or ends with 'C'.

Solution:

```
import pandas as pd

df = pd.DataFrame({
    'Product': ['Apple', 'Banana', 'Apricot', 'Garlic', 'Avocado', 'Carrot'],
    'Price': [50, 30, 45, 20, 60, 25]
})

filtered = df[df['Product'].str.startswith('A') | df['Product'].str.endswith('c')]
print("Filtered results:")
print(filtered)
```

Expected Output:

```
Filtered results:
  Product  Price
0   Apple    50
2  Apricot    45
3   Garlic    20
4  Avocado    60
5   Carrot    25
```

Question 15: Remove Duplicates on Multiple Columns

Difficulty: Intermediate

Remove duplicate rows based on both 'Product' and 'Revenue' columns, keeping the first occurrence.

Solution:

```
import pandas as pd

df = pd.DataFrame({
    'Product': ['A', 'B', 'A', 'B', 'C', 'A'],
    'Revenue': [100, 200, 100, 200, 150, 100],
    'Month': ['Jan', 'Jan', 'Feb', 'Feb', 'Jan', 'Mar']
})

print("Original DataFrame:")
print(df)

df_no_dups = df.drop_duplicates(subset=['Product', 'Revenue'], keep='first')

print("\nAfter removing duplicates:")
print(df_no_dups)
```

Expected Output:

```
Original DataFrame:
   Product  Revenue Month
0        A      100   Jan
1        B      200   Jan
2        A      100   Feb
3        B      200   Feb
4        C      150   Jan
5        A      100   Mar

After removing duplicates:
   Product  Revenue Month
0        A      100   Jan
1        B      200   Jan
4        C      150   Jan
```

Quick Reference Guide

Question	Topic	Concept
Q1	NumPy	Array slicing and reshaping
Q2	NumPy	Percentile calculations
Q3	NumPy	Data normalization
Q4	NumPy	Multidimensional stacking
Q5	NumPy	Unique value counting
Q6	Pandas	Time-based aggregation
Q7	Pandas	Outlier handling
Q8	Pandas	Pivot tables
Q9	Pandas	Missing value imputation

Question	Topic	Concept
Q10	Pandas	Group-based ranking
Q11	Pandas	DataFrame merging
Q12	Pandas	Rolling window functions
Q13	Pandas	Conditional joins
Q14	Pandas	String operations
Q15	Pandas	Duplicate removal

Learning Tips

1. **Understand the "why":** Focus on understanding the logic behind each operation, not just memorizing syntax.
2. **Experiment with variations:** Modify the code—change window sizes, aggregation functions, or filter conditions.
3. **Combine concepts:** Try using multiple techniques in a single query (e.g., groupby + pivot + filtering).
4. **Profile your code:** For large datasets, use `%%timeit` in Jupyter to compare different approaches.
5. **Read error messages carefully:** They often point to data type mismatches or missing values.
6. **Practice with real datasets:** Download datasets from Kaggle or use publicly available APIs to work with realistic data.

Intermediate NumPy & Pandas Questionnaire

Created: November 12, 2025

Suitable for: Intermediate learners with basic Python and data structures knowledge